ETAPA 2 - DESENVOLVIMENTO E INTEGRAÇÃO PARCIAL DOS MÓDULOS

EMBARCATECH

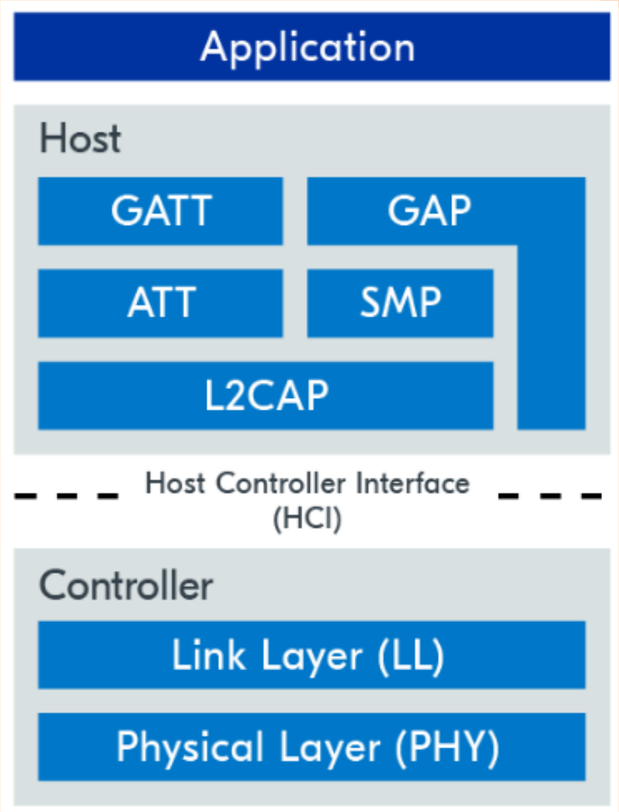
Amigo Perto

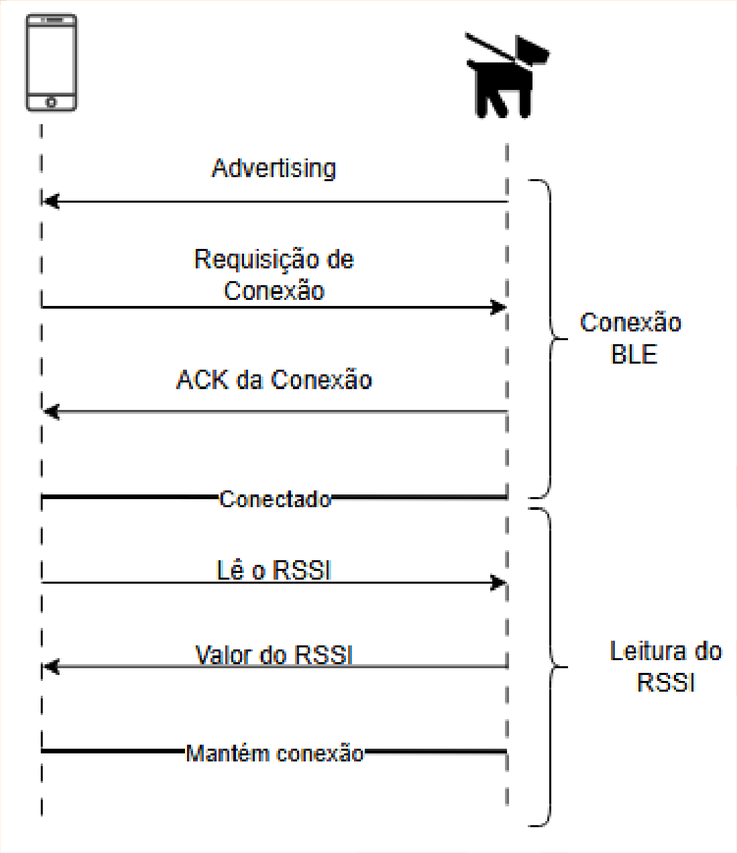
ANTONIO, ERIC E VITOR

SEMANA 1

ESTUDO DO BLUEETOOTH/ SDK

Função / API	Módulo (Zephyr / SDK)	O que faz no código
bt_enable()	Bluetooth Core	Inicializa toda a pilha Bluetooth
bt_set_name()	GAP	Define o nome da coleira
bt_le_adv_start()	GAP Advertising	Começa o advertising com a coleira em modo conectável
bt_conn_cb_register()	Connection	Registra callbacks de conexão e desconexão
bt_gatt_service_register()	GATT Server	Registra o serviço GATT customizado com características
bt_conn_unref()	Connection	Libera referência da conexão quando desconecta
bt_gatt_attr_read()	GATT Server	Função callback quando o app lê uma característica
bt_gatt_attr_write()	GATT Server	Callback quando o app envia comando (ex: beep, novo limite etc.)

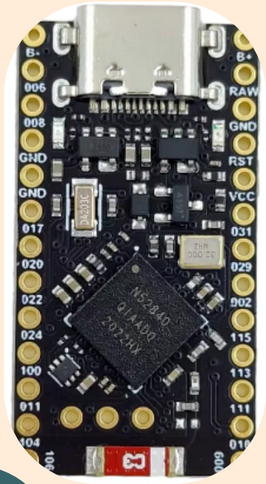




nRF54L15

- Ausência de interface USB para programação direta.
- Programador SWD adquirido apresentou incompatibilidade e não funcionou.
- Processo de programação mostrou-se complexo e pouco prático.
- Apesar dos desafios, a placa utiliza um firmware mais recente, facilitando a integração com o aplicativo móvel.
- Possui Buzzer, LED e suporte para bateria integrados.

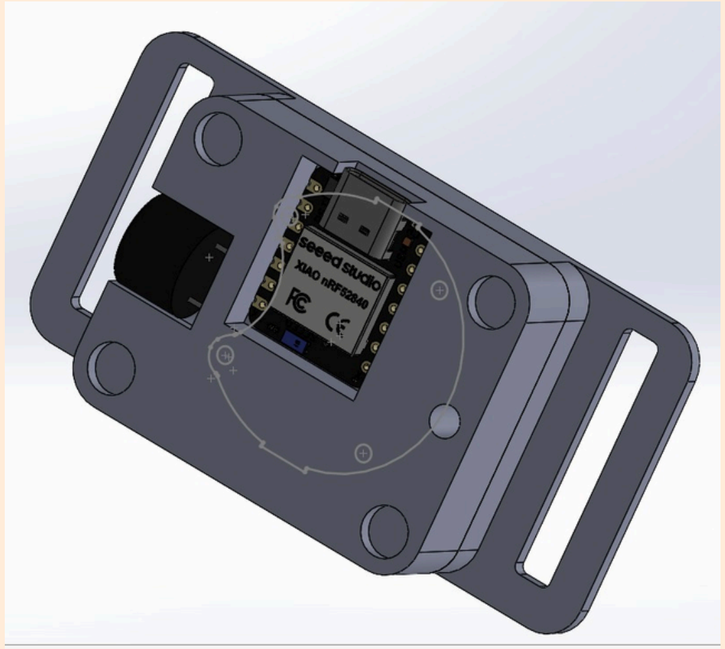
nRF52840

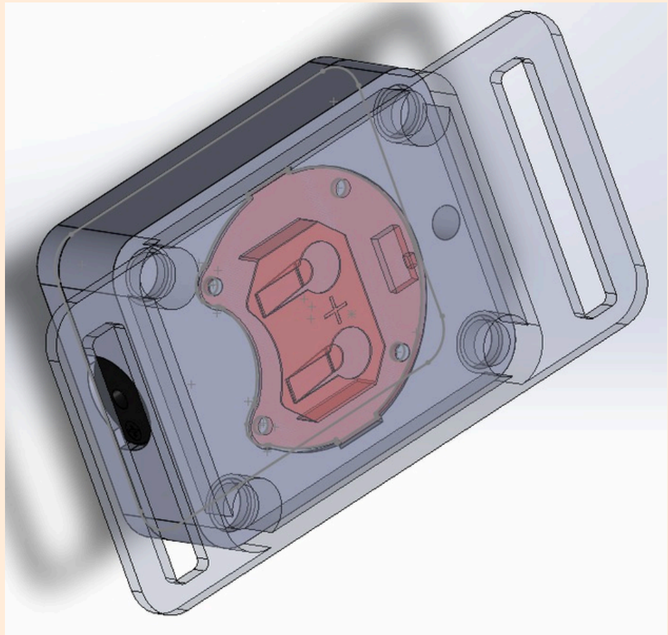


- Programação significativamente mais simples devido à interface USB.
- Maior consumo de energia e firmware menos compatível para integração com o aplicativo móvel.
- Não possui buzzer, LED ou suporte para bateria integrados.
- Opção mais favorável no momento pela facilidade de programação, embora ainda seja necessário avaliar cuidadosamente os prós e contras antes da decisão final.

SEMANA 3

CASES PARA A COLEIRA





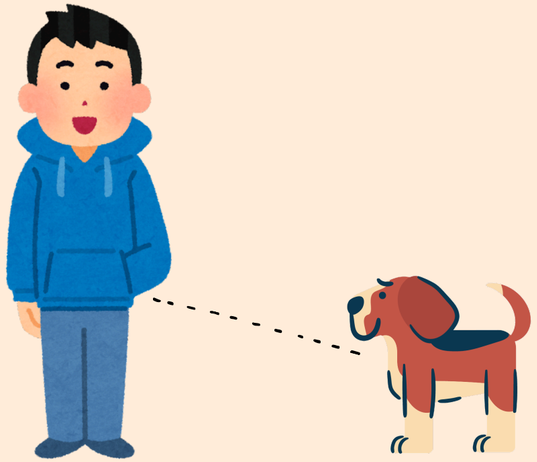
SEMANA 1

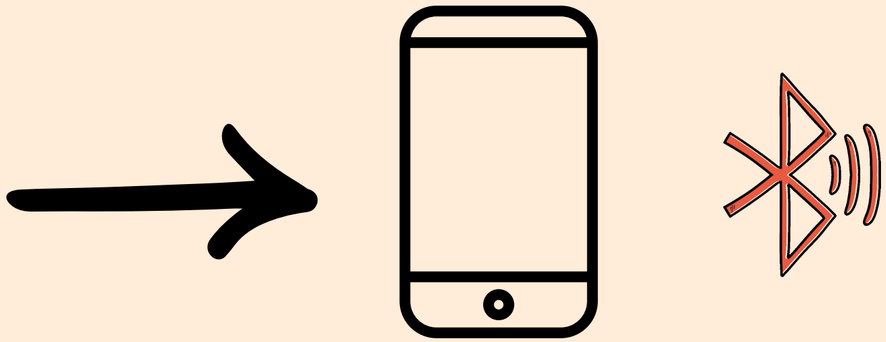
ESTUDO DO APLICATIVO

Categoria	Biblioteca / Framework	Função / Uso no Projeto	Observações
BLE Core	BluetoothAdapter, BluetoothManager	Habilitar Bluetooth, iniciar scan	API nativa Android
BLE Scan	BluetoothLeScanner, ScanFilter, ScanSettings	Escanear dispositivos BLE com filtro de	Requer permissões
Conexão BLE	BluetoothGatt, BluetoothGattCallback	Conectar ao dispositivo BLE	Base nativa, baixo nível
Nordic BLE Library	no.nordicsemi.android:ble-ktx	Facilita conexão BLE, notificações, leituras e escritas	Recomendado pela Nordic, alto nível
MVVM	Android Architecture Components	Separar UI / lógica	Padrão oficial Android
UI	Jetpack Compose **ou** XML	Interface do app	Compose recomendado
Permissões	ActivityResultContracts, Manifest.permission.*	Pedir permissões BLE e localização	Obrigatório Android 12+
Logs	Timber	Logging organizado	Opcional
Persistência	DataStore	Guardar último dispositivo conectado	Seguro e moderno
Serialização	Kotlin Serialization	Enviar JSON no BLE-UART (opcional)	Para payloads estruturados

Componente	Função	Responsabilidade
Android → scanDevices()	Escanear BLE	Localizar nRF
Android → connect(device)	Conectar ao peripheral	Iniciar GATT
Android → writeCharacteristic(uuid, data)	Enviar comando (LED/vibração)	Controle do dispositivo
Android → enableNotifications(uuid)	Receber dados	Status, bateria
Firmware → write_cb()	Receber comando	Acionar GPIO/PWM
Firmware → bt_gatt_notify()	Enviar eventos ao Android	Mudança de estado
Firmware → gpio_pin_set()	Acionar carga	LED/relé
Firmware → pwm_set()	Vibrar/ajustar intensidade	Motores/sinalização

SEMANA 2





DEMONSTRAÇÃO DO APP

Web | bluetooth.service.ts 7

src > app > bluetooth.service.ts > ...

```
export interface Device {
  name: string;
  id: string;
  rssi?: number;
  distance?: number;
  distanceCategory?: string;
}

// --- Constantes de Rastreamento e Alerta ---
const IMMEDIATE_ALERT_SERVICE_UUID = '00001802-0000-1000-8000-00805f9b34fb';
const ALERT_LEVEL_CHARACTERISTIC_UUID = '00002a06-0000-1000-8000-00805f9b34fb';

// --- Constantes de Calibração de Distância (RSSI) ---
const MEASURED_POWER_AT_1M = -52; // Potência do sinal (em dBm) medida a 1 metro de distância.
const ENVIRONMENTAL_FACTOR = 4;   // Fator ambiental. Varia de 2 (espaço aberto) a 4 (ambientes internos com paredes).
const RSSI_OUT_OF_RANGE_THRESHOLD = -100; // Limite de RSSI para considerar "fora de alcance".


export type OperatingMode = 'idle' | 'radar' | 'alert';

@Injectable({
  providedIn: 'root',
})
export class BluetoothService {
  private zone = inject(NgZone);
  private bluetoothDevice: BluetoothDevice | null = null;
  private alertLevelCharacteristic: BluetoothRemoteGATTCharacteristic | null = null;
  private watchAdvertisementsController: AbortController | null = null;
  private audioContext: AudioContext | null = null;
  private alertIntervalId: any = null;

  // --- Sinais Públicos de Estado ---
  device = signal<Device | null>(null);
  operatingMode = signal<OperatingMode>('idle');
```

SEMANA 3

TESTES DE INTEGRAÇÃO

ID	Cenário	Resultado Esperado	Resultado Observado	Status
INT-01	Descoberta de Dispositivo (Radar)	App entra no modo radar e exibe o dispositivo encontrado.	Scan realizado, dispositivo detectado e UI atualizada.	**Aprovado**
INT-02	Ativação do Modo Alerta	Exibir spinner, conectar e mudar para estado alert.	Spinner OK, conexão bem-sucedida e UI atualizada.	**Aprovado**
INT-03	Proximidade — Fora de Alcance	Exibir alerta visual “Dispositivo fora de alcance!”.	Desconexão detectada e barra de alerta exibida corretamente.	**Aprovado**
INT-04	Botão de Pânico	Obter coordenadas e abrir Google Maps com os parâmetros.	Coordenadas obtidas e URL aberta corretamente.	**Aprovado**
INT-05	Erro — Permissão de Localização Negada	Exibir mensagem de erro de permissão.	Erro capturado e mensagem exibida na UI.	**Aprovado**
INT-06	Erro — Falha na Conexão BLE	Exibir erro e encerrar loading.	Erro capturado, signals atualizados e UI notificada.	**Aprovado**

OBRIGADO!

DÚVIDAS?